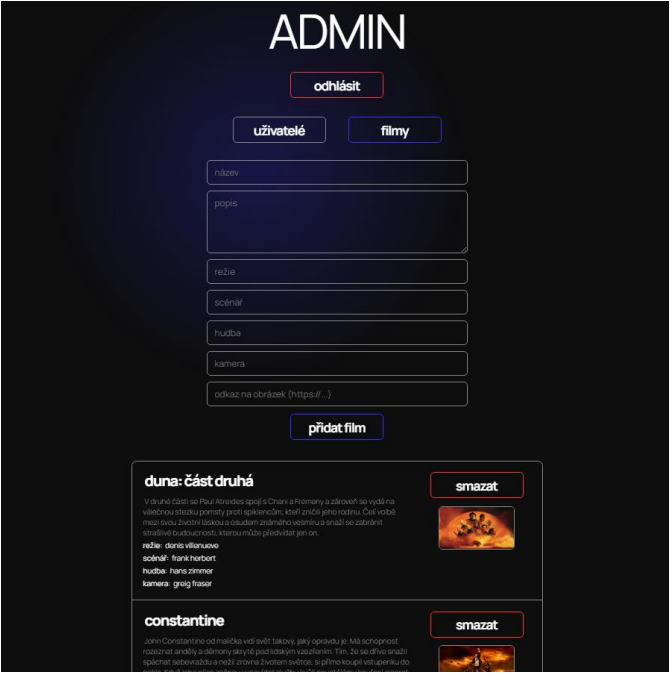
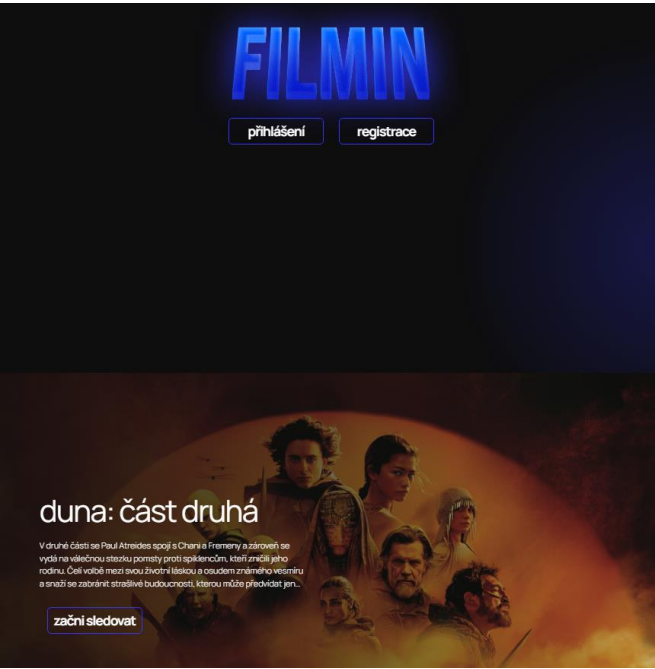


Ukázka aplikace



FILMIN – DOKUMENTACE

Jiří Kubík

Úvod

Filmin je webová aplikace, která umožňuje uživatelům registraci, výběr předplatného a procházení katalogu filmů.

Administrátor má k dispozici samostatné rozhraní pro správu uživatelů (změna plánu, mazání účtů) a správu filmů (vytváření, úprava, mazání).

Použité technologie

Backend

- Programovací jazyk: Java 17
- Build: Maven
- Framework: Spring Boot
- Bezpečnost: Spring Security, JWT
- Persistence dat: Spring Data JPA, Hibernate
- Databáze: H2
- Dokumentace API: OpenAPI, Swagger

Frontend

- HTML
- CSS
- JavaScript
- React
- Vite

Ostatní

- Figma
- ChatGPT
- Claude
- DevTools

Architektura aplikace

Aplikace je rozdělena na frontend a backend.

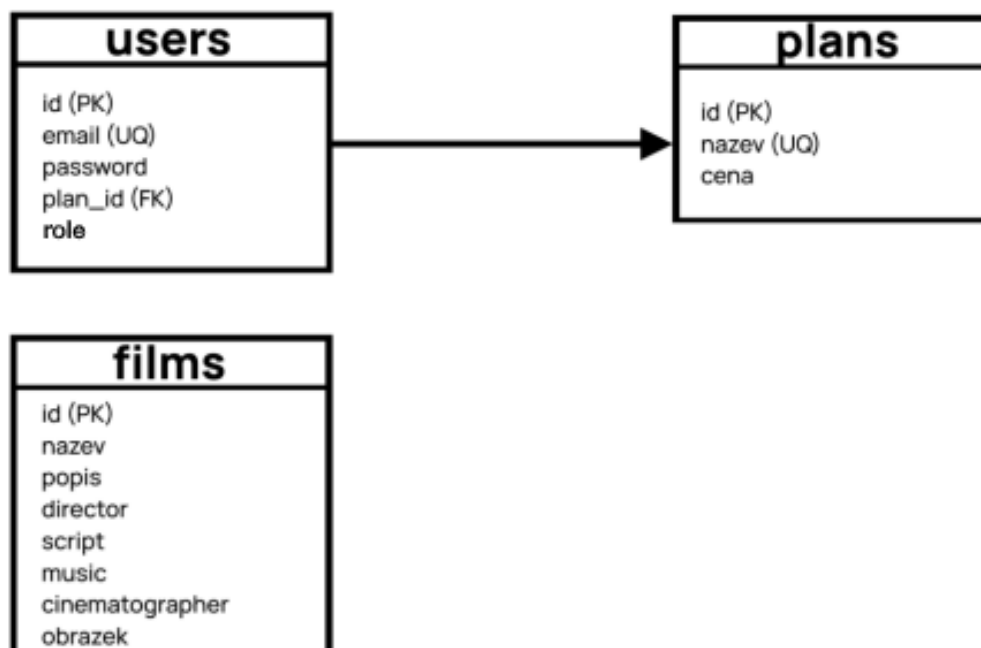
Jelikož frontend používá React, který umožňuje tvorbu komponent, je velká část frontendu tvořena tak, aby byly jednotlivé části co nejvíce znovupoužitelné (například tlačítka, navbar).

Backend dodržuje klasickou vícevrstvou architekturu, kde každá vrstva má pevně daný svůj účel a komunikuje pouze se sousedními vrstvami.

Konkrétní balíčky v projektu:

- **controller – přijímá HTTP požadavky, validuje vstup, předává data dál**
- **service – obsahuje business logiku a transakční hranice**
- **repository – přístup k databázi přes Spring Data JPA, vlastní JPQL dotazy**
- **dto – entity i přenosové objekty**
- **security – konfigurace Spring Security, JWT generace a validace**
- **configuration – ostatní konfigurační beany (Swagger, inicializace admina)**

Datový model



Bezpečnost a autentizace

Autentizace funguje na principu JWT (JSON Web Token). Po úspěšném přihlášení server vrátí token, který si klient uloží do localStorage a posílá ho v hlavičce Authorization: Bearer <token> u všech chráněných requestů. Token nese identitu uživatele, jeho roli a expiraci, a je podepsán tajným klíčem definovaným v application.properties.

V systému jsou dvě role – USER (běžný uživatel) a ADMIN (administrátor). Role je uložena přímo v databázi v tabulce users a její kontrolu zajišťuje Spring Security konfigurace (SecurityConfig.java), která chrání všechny admin endpointy.

Dokumentace API

Filmy Správa filmových titulu (cteni veřejně, modifikace pouze admin)			^
GET	/api/films/{id}	Detail filmu podle ID	🔒 ▼
PUT	/api/films/{id}	Upravi existující film	🔒 ▼
DELETE	/api/films/{id}	Smaze film podle ID	🔒 ▼
GET	/api/films	Vrati seznam všech filmů	🔒 ▼
POST	/api/films	Vytvoří nový film	🔒 ▼
GET	/api/films/search	Vyhledá filmy podle textu	🔒 ▼
Admin Správa uživatele - vyzaduje JWT token s rolí ADMIN.			^
POST	/api/admin/users/{id}/change-plan		🔒 ▼
POST	/api/admin/login		🔒 ▼
GET	/api/admin/users		🔒 ▼
GET	/api/admin/session-check		🔒 ▼
DELETE	/api/admin/users/{id}		🔒 ▼
Uživatelé - autentizace Přihlášení, informace o účtu, změna plánu, smazání účtu			^
POST	/api/users/logout		🔒 ▼
POST	/api/users/login	Přihlášení uživatele	🔒 ▼
POST	/api/users/change-plan		🔒 ▼
GET	/api/users/user-info		🔒 ▼
DELETE	/api/users/delete-account		🔒 ▼
user-controller			^
GET	/api/users/{id}		🔒 ▼
PUT	/api/users/{id}		🔒 ▼
DELETE	/api/users/{id}		🔒 ▼
POST	/api/users		🔒 ▼
plan-controller			^
GET	/api/plans		🔒 ▼
GET	/api/plans/{id}		🔒 ▼

Validace

Validace vstupů

- DTO třídy obsahují anotace **@NotBlank**, **@NotNull** apod.
- Controllery používají **@Valid** před parametrem **@RequestBody**
- Při validační chybě se vrací **HTTP 400**

HTTP kód	Význam
200 OK	Úspěch (vrací data)
201 Created	Úspěšné vytvoření (POST)
400 Bad Request	Validační chyba, neplatný vstup
401 Unauthorized	Chybí nebo neplatný JWT token / špatné heslo
403 Forbidden	Token je validní, ale role nestačí
404 Not Found	Entita neexistuje